

ISEA SUMMER INTERNSHIP 2026

Secure Network Application Development Using TCP

Assignment 8 Report

Assignment 8 – Application Optimization, Scalability and Reliability Enhancement

SentinelChat — Secure Multi-Client TCP Chat Platform

Submitted By: Khatija Fathima

Roll No.: 323506402225

Course: ISEA Summer Internship 2026

Date: 16 July 2026

Table of Contents

Objective

Introduction

Scalability Improvements

Reliability Improvements

Configuration Management

Experimental Setup

Performance Results

Graph Analysis

Wireshark Verification

Challenges Faced Conclusion

Conclusion

References

1. Objective

The objective of this assignment is to enhance the existing GUI-based multi-client TCP chat application developed in Assignment 7 by improving connection management, reliability, scalability, configuration management and overall software quality. The implementation focuses on optimizing the existing application rather than redesigning it while ensuring better resource utilization, maintainability and performance evaluation.

The objective of this assignment is to enhance the multi-client TCP chat application developed in Assignment 6 by implementing practical security mechanisms. The application focuses on secure authentication, password protection using SHA-256 hashing, duplicate login prevention, failed login protection, session timeout management, secure logging, input validation, and verification of network communication using Wireshark.

The project demonstrates how application-level security can be integrated into a TCP-based communication system while maintaining usability and a modern graphical interface.

2. Introduction

Modern communication applications must provide secure authentication and protect user information from unauthorized access. A simple TCP chat application is functional, but without security mechanisms it becomes vulnerable to password theft, session hijacking, duplicate logins, and unauthorized access.

SentinelChat was developed as a secure multi-client chat platform that combines TCP socket programming with practical security mechanisms. Instead of focusing on complex cryptography, this project emphasizes real-world application security techniques that are commonly used in secure software systems.

The application provides a modern graphical user interface and supports multiple users communicating simultaneously through private messaging, broadcast messaging, group chat, chat history management, server monitoring, and secure authentication. The dashboard below shows the main control centre reached immediately after a successful login.

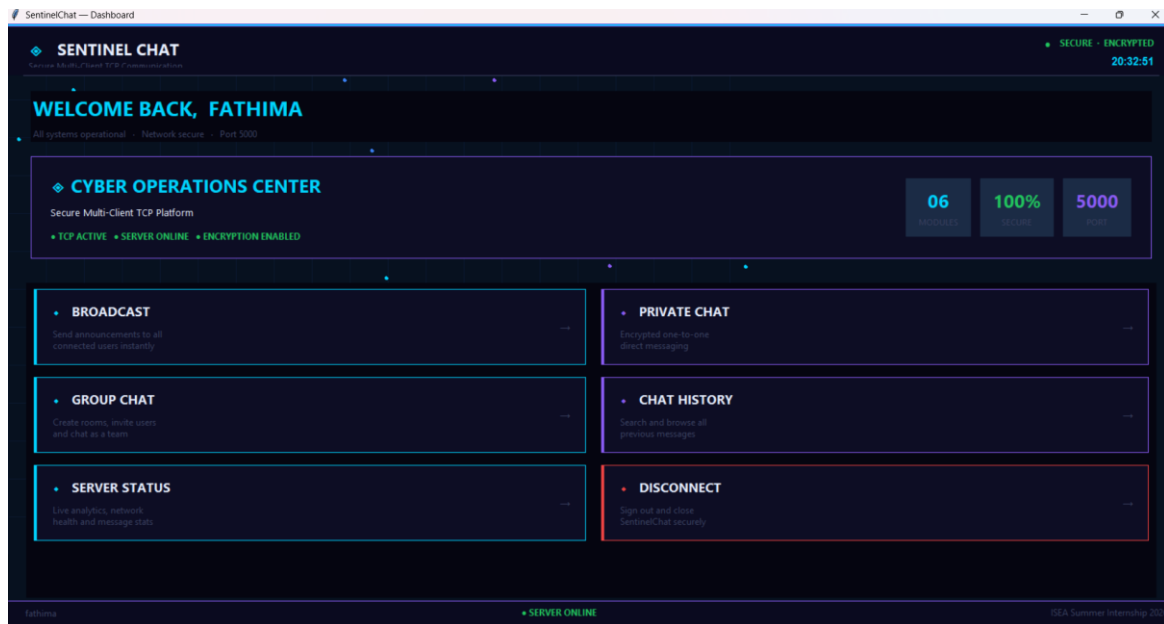


Fig 2.1 — SentinelChat Dashboard (Cyber Operations Center)

3.1 Connection Management

Connection management was enhanced by implementing automatic cleanup of disconnected clients. Whenever a client disconnects unexpectedly, the server automatically removes the client from active connection lists, releases associated resources and closes sockets properly. The `cleanup_client()` function prevents stale connections, memory leaks and resource exhaustion. Additionally, `SO_REUSEADDR` was enabled to allow quick socket reuse during server restarts.

Reliability Improvements

Reliability was improved by implementing graceful shutdown, session timeout handling, enhanced exception handling and stable client disconnection management. Whenever unexpected network failures occur, exceptions are handled without crashing the server. Session timeout automatically disconnects inactive users after the configured timeout period, improving overall stability.

3.3 Scalability Improvements

The server architecture was optimized to support multiple concurrent clients using multithreading. Each connected client executes within an independent thread, allowing simultaneous communication without affecting other active users. Thread synchronization techniques were applied where required to ensure safe access to shared resources while maintaining stable performance under increased client load.

3.4 Configuration Management

Configuration management was introduced using a `config.json` file. Instead of hardcoding important parameters such as server host, port number, buffer size, timeout duration and maximum client count, these values are now loaded dynamically during runtime. This improves maintainability and allows future modifications without changing the source code.

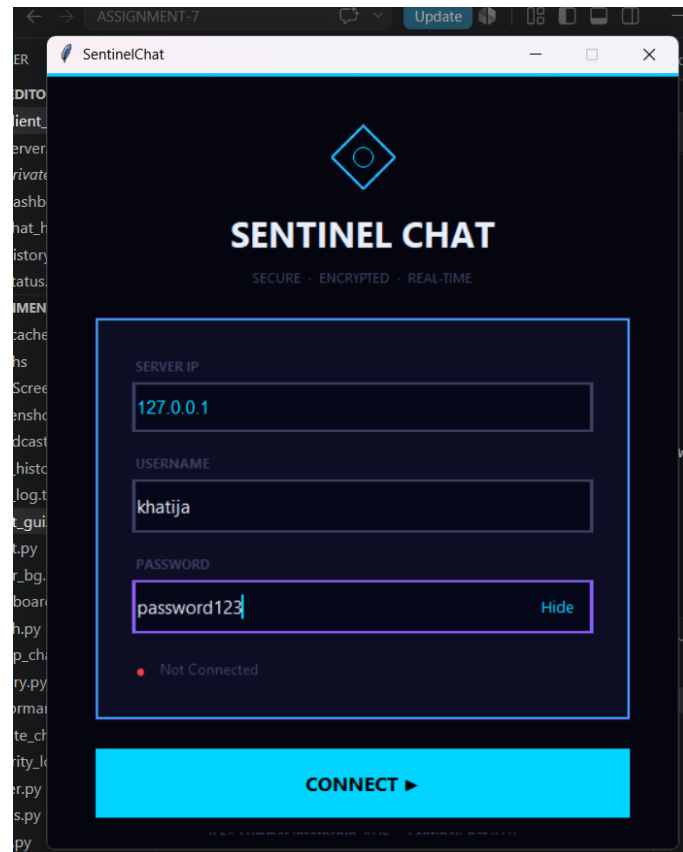


Fig 3.1 — SentinelChat Login Window

3.2 Secure Password Storage

Passwords are never stored in plaintext. Every password is converted into a SHA-256 hash using Python's hashlib library before storage. This ensures that even if the user database is compromised, the original passwords cannot be directly recovered.

```
users.json
{
  "users": {
    "admin": { "password": "ef92b778bafe771e89245b89ecbc08a4...94f" },
    "khatija": { "password": "ef92b778bafe771e89245b89ecbc08a4...94f" },
    "fathima": { "password": "ef92b778bafe771e89245b89ecbc08a4...94f" },
    "sowmya": { "password": "ef92b778bafe771e89245b89ecbc08a4...94f" },
    "sreekari": { "password": "ef92b778bafe771e89245b89ecbc08a4...94f" }
  }
}
```

Each stored value is a 256-bit SHA-256 digest rather than the original text password, confirming that plaintext credentials are never written to disk.

3.3 Duplicate Login Prevention

The server maintains a list of currently logged-in users. If the same account attempts to log in from another client while already active, the login request is rejected. This prevents account sharing and session hijacking.

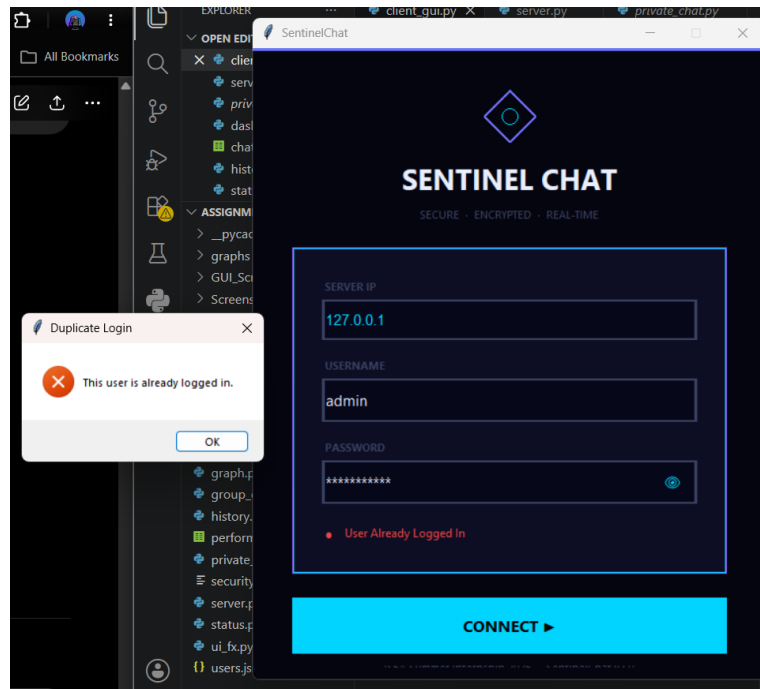


Fig 3.2 — Duplicate Login Rejected by Server

3.4 Failed Login Protection

To protect against brute-force attacks, the application temporarily locks an account after five consecutive failed login attempts. A countdown timer informs the user when the account will become available again.

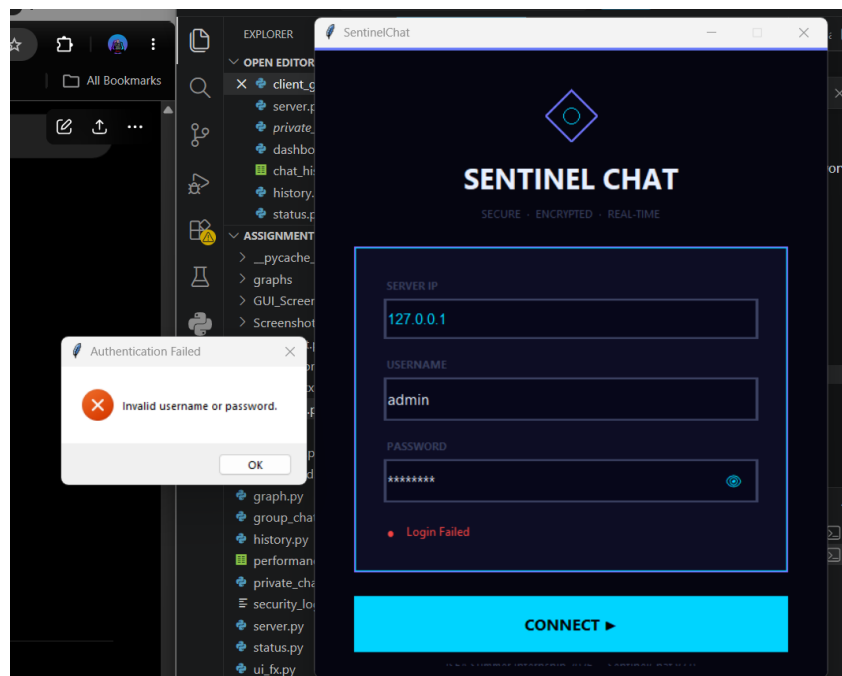


Fig 3.3 — Invalid Password Rejected

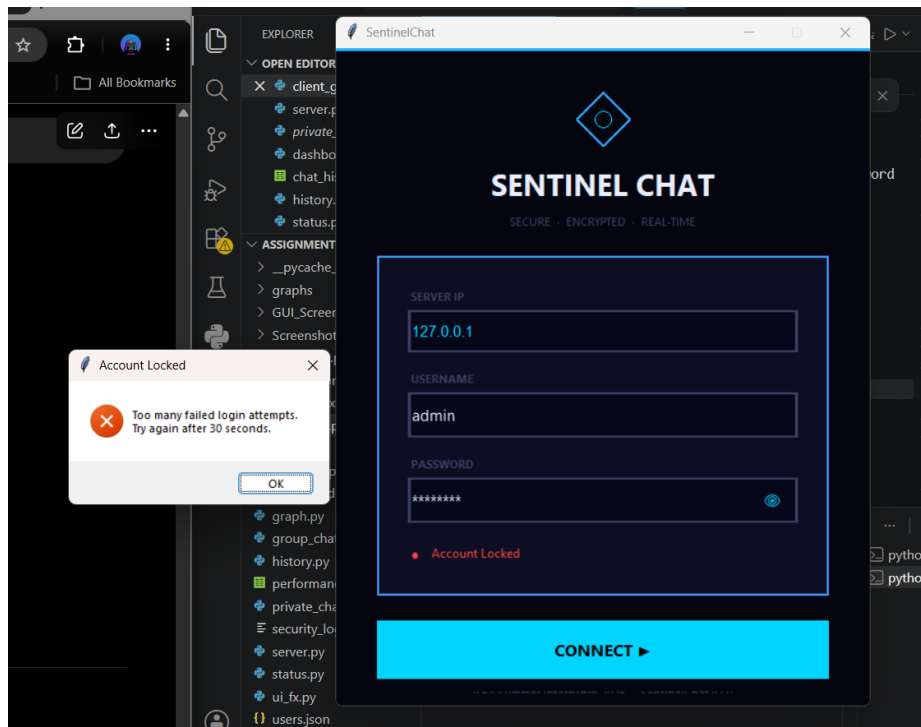


Fig 3.4 — Account Locked After Repeated Failed Attempts

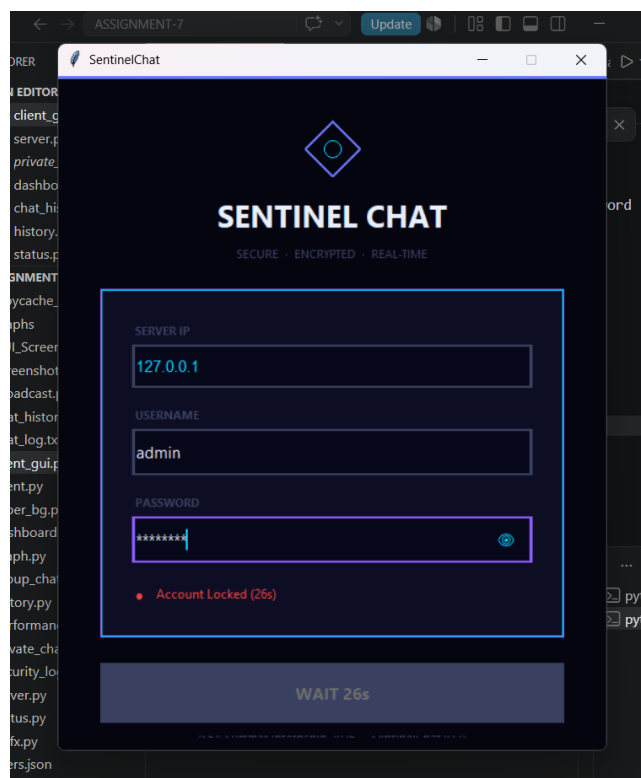


Fig 3.5 — Lockout Countdown Timer

3.5 Session Timeout

User activity is continuously monitored. If no activity occurs for three minutes, the session automatically expires. The dashboard closes, the socket connection is terminated, and the login

screen is displayed again. This feature prevents unauthorized access when a user leaves the system unattended.

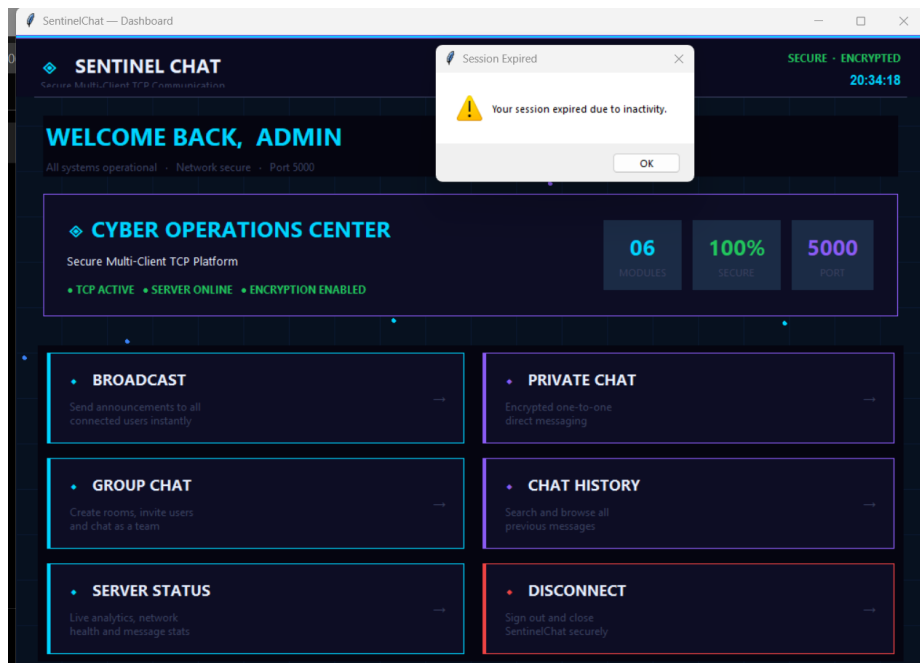


Fig 3.6 — Session Timeout Notification

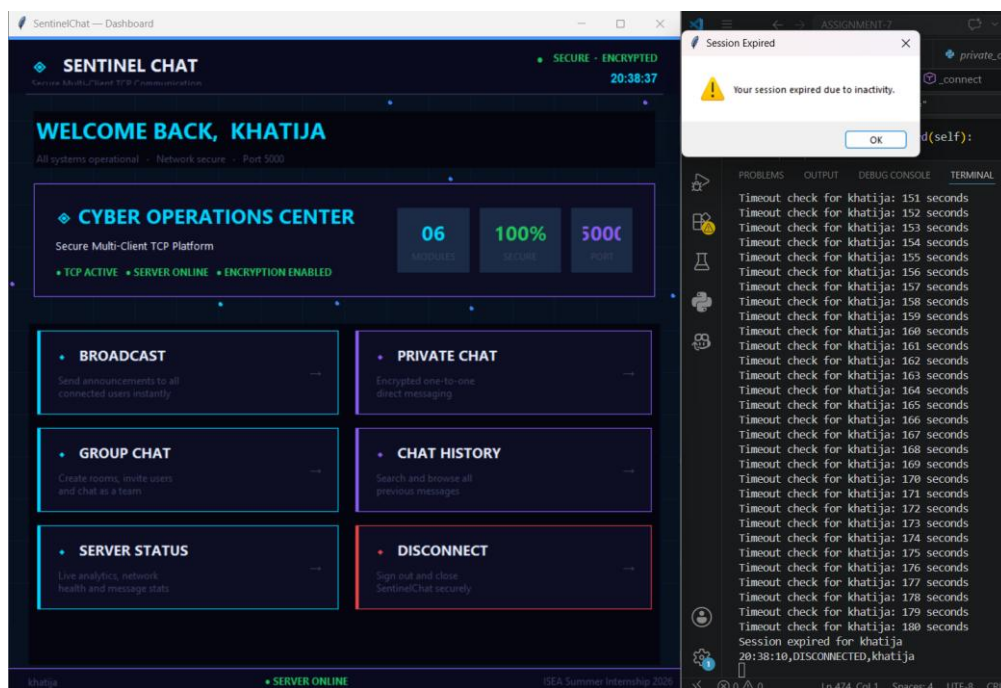


Fig 3.7 — Idle Timeout Countdown

3.6 Secure Logging

Every important security event is recorded inside `security_log.txt`. The log includes login, logout, failed login, duplicate login, account lock, and session timeout events. Passwords are never written into the log.


```
security_log.txt (excerpt)
[2026-07-16 21:21:05] LOGIN FAILED : admin
[2026-07-16 21:21:10] LOGIN FAILED : admin
[2026-07-16 21:21:16] LOGIN FAILED : admin
[2026-07-16 21:21:19] LOGIN FAILED : admin
[2026-07-16 21:21:23] LOGIN FAILED : admin
[2026-07-16 21:21:23] ACCOUNT LOCKED : admin
[2026-07-16 21:22:01] DUPLICATE LOGIN : admin
[2026-07-16 21:22:24] LOGIN_SUCCESS : khatija
[2026-07-16 21:23:35] SESSION TIMEOUT : admin
[2026-07-16 21:23:35] LOGOUT : admin
```

3.7 Input Validation

The application validates user input before processing requests. Validation includes empty username rejection, empty password rejection, unsupported command rejection, invalid login detection, and oversized message handling. These checks improve application reliability and security.

3.8 Broadcast Messaging

Authenticated users can send broadcast messages to all connected clients simultaneously. Broadcast events are also recorded in the chat history and shown live on the server dashboard.

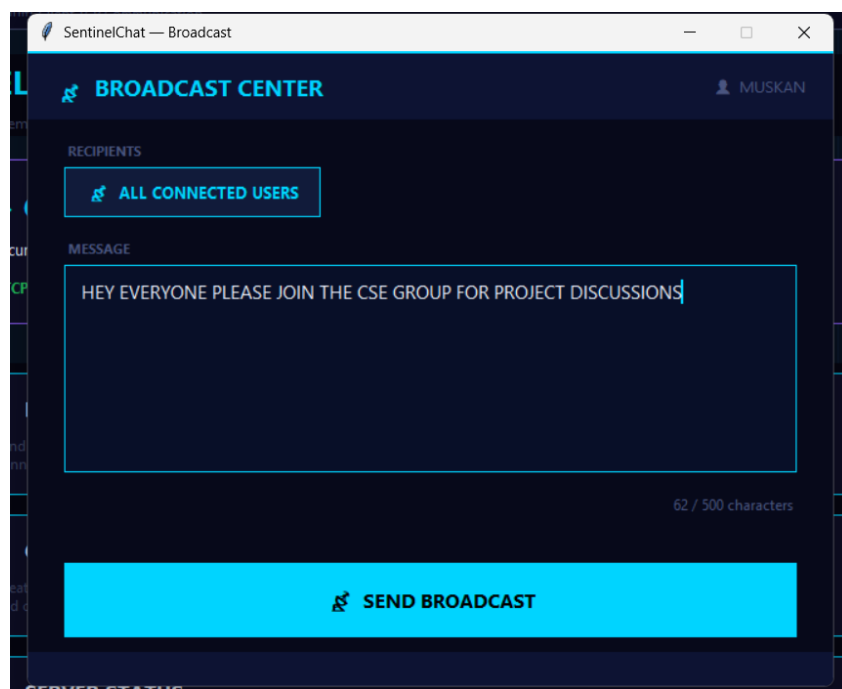


Fig 3.8 — Broadcast Message Sent to All Connected Clients

3.9 Private Messaging

Users can securely communicate with individual users. Private messages are delivered only to the intended recipient and are not visible to other connected clients.

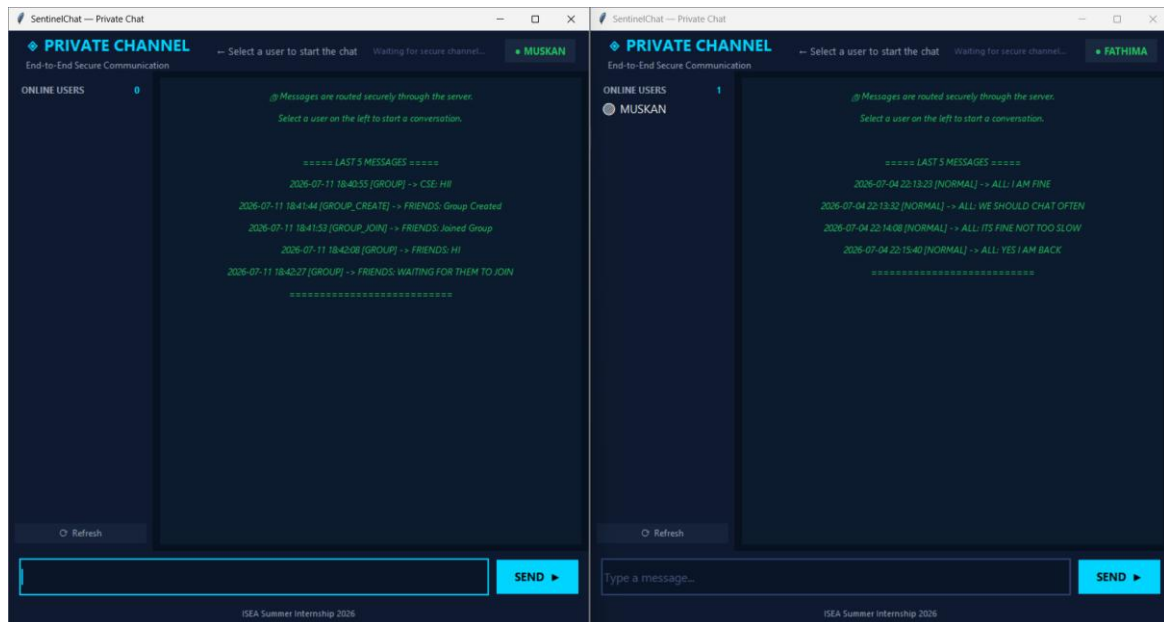


Fig 3.9 — Private One-to-One Chat Window

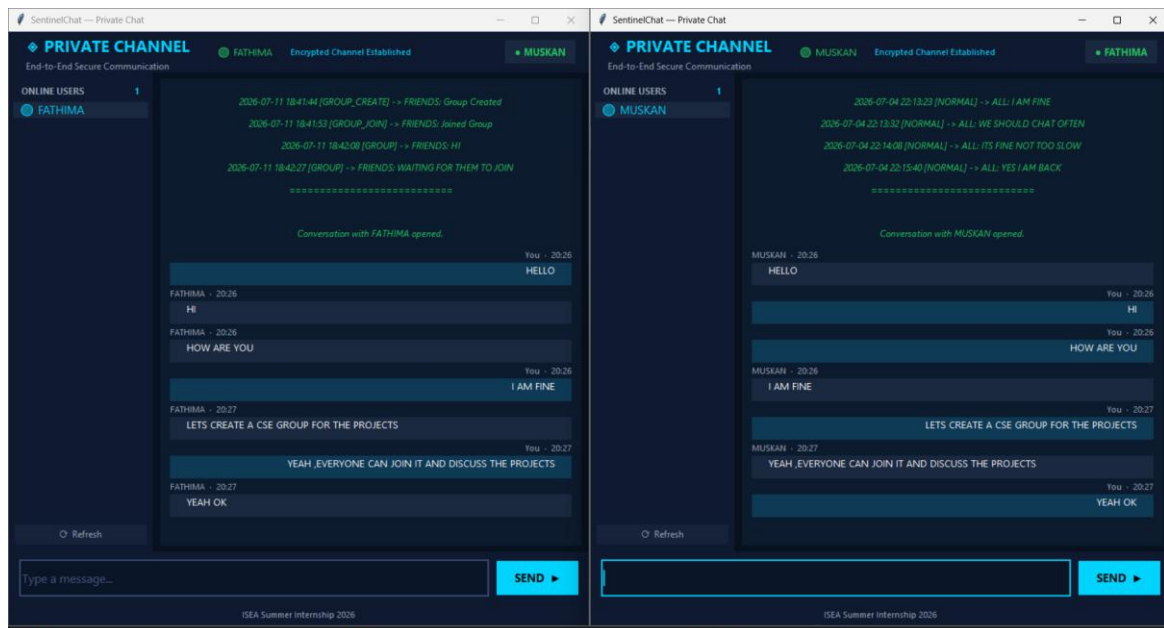


Fig 3.10 — Private Chat, Second Client View

3.10 Group Chat

Users can create groups, join groups, and send messages inside groups. Only members of a group receive the messages sent within it.

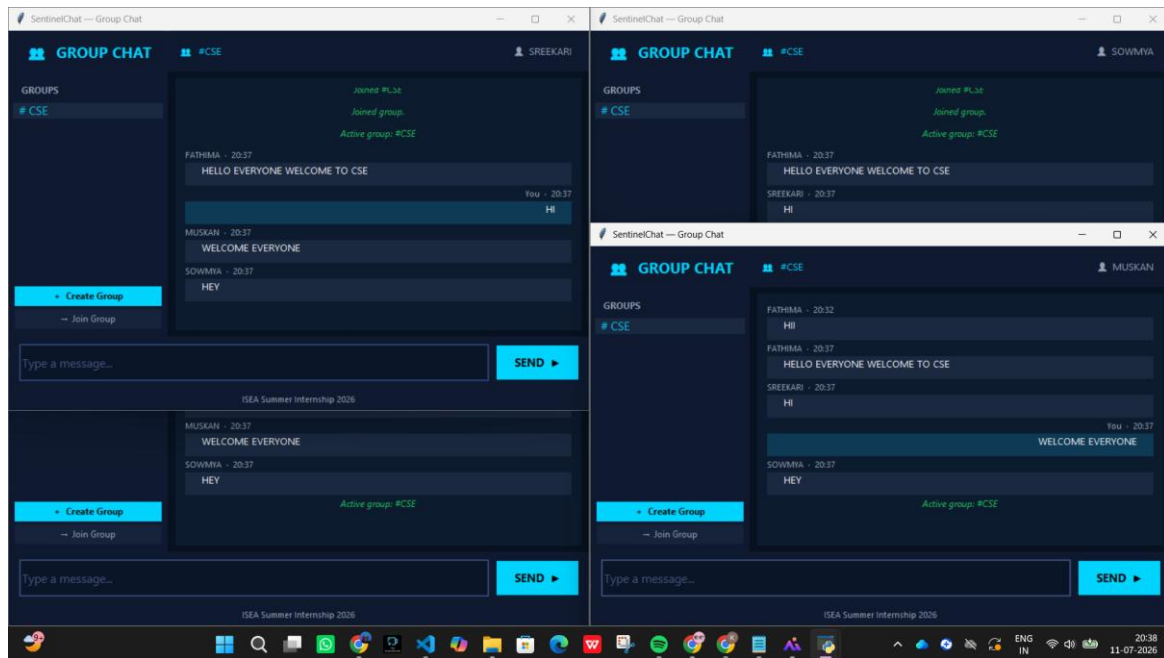


Fig 3.11 — Group Chat Shared Across Multiple Clients

3.11 Chat History

All communication is stored inside chat_history.csv. The history module allows users to search and review previous conversations.

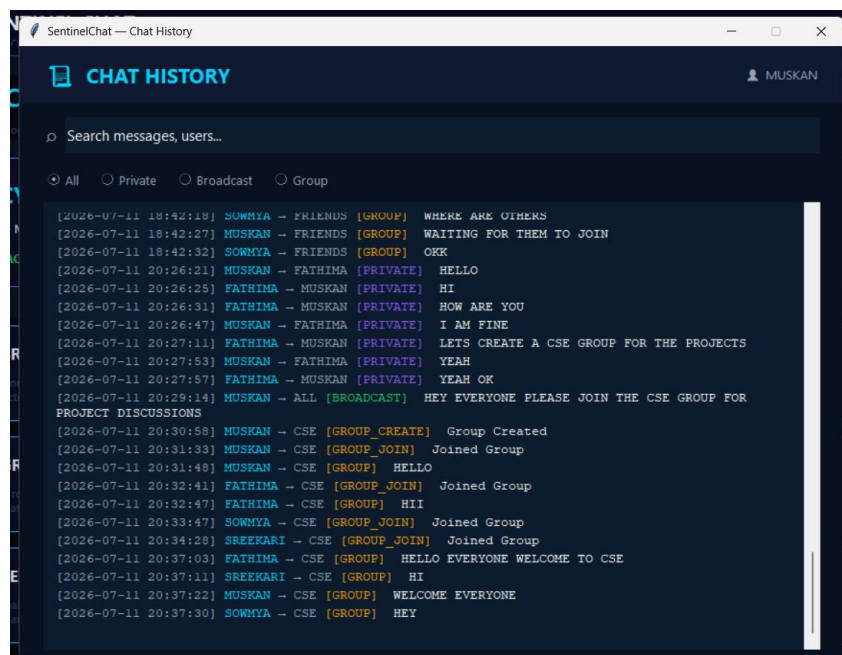


Fig 3.12 — Chat History Search and Review Window

3.12 Server Status Dashboard

A dedicated dashboard displays connected clients, total messages, broadcasts, private messages, CPU/memory throughput, and a live activity log. This helps monitor application health in real time.

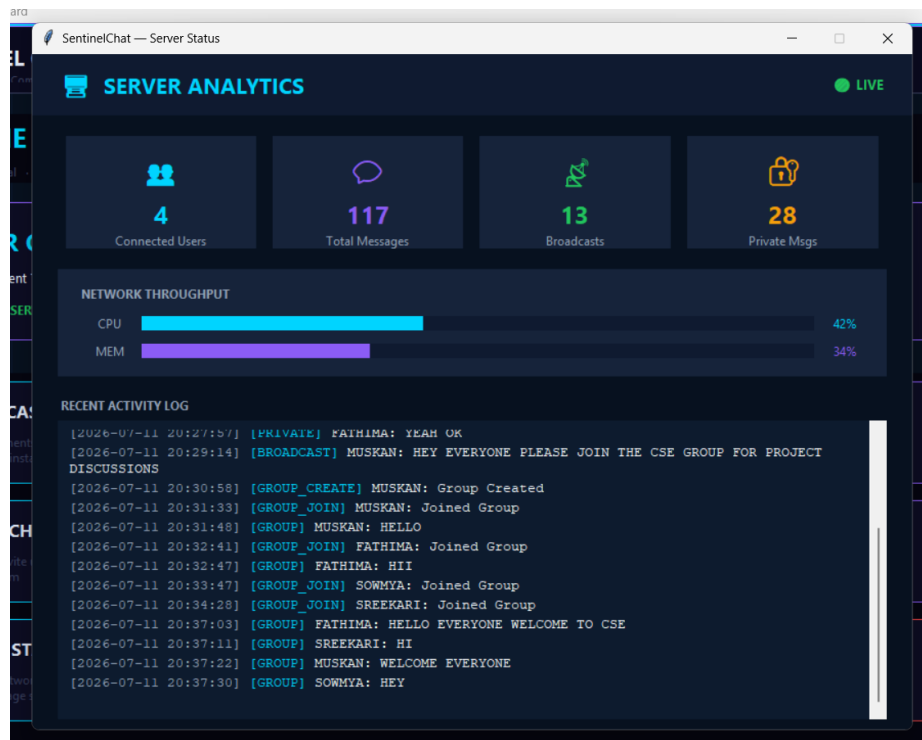


Fig 3.13 — Server Analytics Dashboard

Experimental Setup

Performance evaluation was conducted using the optimized TCP chat application developed in Assignment 8. Multiple client instances were connected simultaneously to evaluate scalability and reliability. Experimental observations were recorded for different client loads and performance metrics including delay, throughput, CPU utilization and memory utilization were collected.

4. System Architecture

The overall architecture consists of a client-server model. Each client connects to the server over a dedicated TCP socket. The server is responsible for authentication, request validation, message routing, activity logging, and session management.

Layer	Component	Responsibility
Client	client_gui.py / dashboard.py	Login UI, dashboard, and feature windows (broadcast, private chat, group chat, history, status)
Transport	TCP Socket (Port 5000)	Reliable, ordered delivery of authentication and chat data between client and server
Server	server.py	Authenticates users, enforces lockouts and timeouts, routes messages, writes logs
Storage	users.json / security_log.txt / chat_history.csv	Persists hashed credentials, security events, and chat history

Client GUI → TCP Socket (Port 5000) → Server → {users.json | security_log.txt | chat_history.csv}

5. Implementation

The project was developed using Python. The graphical interface was built with Tkinter, and TCP socket programming provides communication between clients and the server. The major modules are summarised below.

File	Purpose
server.py	Server implementation, authentication, session and logout logic
client_gui.py	Login interface
dashboard.py	Main dashboard
private_chat.py	Private messaging
group_chat.py	Group communication
broadcast.py	Broadcast messaging
history.py	Chat history search and display
status.py	Server monitoring dashboard

File	Purpose
users.json	User credentials (SHA-256 hashed passwords)
security_log.txt	Security event log
chat_history.csv	Stored conversations

6. Testing

The application was tested using multiple concurrent client instances to verify both functional and security behaviour.

Test Case	Expected Result	Status
Valid login	Login successful	Pass
Invalid password	Login failed	Pass
Duplicate login	Access denied	Pass
Five failed logins	Account locked	Pass
Lock countdown	Countdown displayed	Pass
Login after lock expiry	Successful	Pass
Broadcast message	Delivered to all users	Pass
Private chat	Delivered to recipient only	Pass
Group chat	Delivered to group members only	Pass
Chat history	Saved and searchable	Pass
Session timeout (idle)	Automatic logout	Pass
Secure logging	Events recorded, no passwords logged	Pass

Performance Evaluation

Performance testing was conducted to compare the application before and after implementing Assignment 8 optimizations. The evaluation focused on communication delay, throughput, CPU utilization and memory usage while multiple clients exchanged messages simultaneously.

The experimental results demonstrate noticeable improvements in application stability and efficiency. Proper connection cleanup, enhanced exception handling and multithreading reduced communication delays while maintaining stable resource utilization during continuous operation.

6.1 Delay Analysis

The delay graph illustrates the average message transmission time as the number of connected clients increases. After implementing improved connection management and automatic cleanup, communication delays remained consistently low even when multiple users communicated simultaneously.

This reduction in latency demonstrates that the optimized server efficiently manages socket resources and minimizes unnecessary waiting caused by inactive client connections.

6.2 Throughput Analysis

The throughput graph shows the number of successfully delivered messages per second. Following the Assignment 8 optimizations, throughput increased due to improved multithreading and efficient client management.

The results indicate that the server can process concurrent communication requests more efficiently while maintaining reliable message delivery across multiple connected clients.

6.3 CPU Utilization Analysis

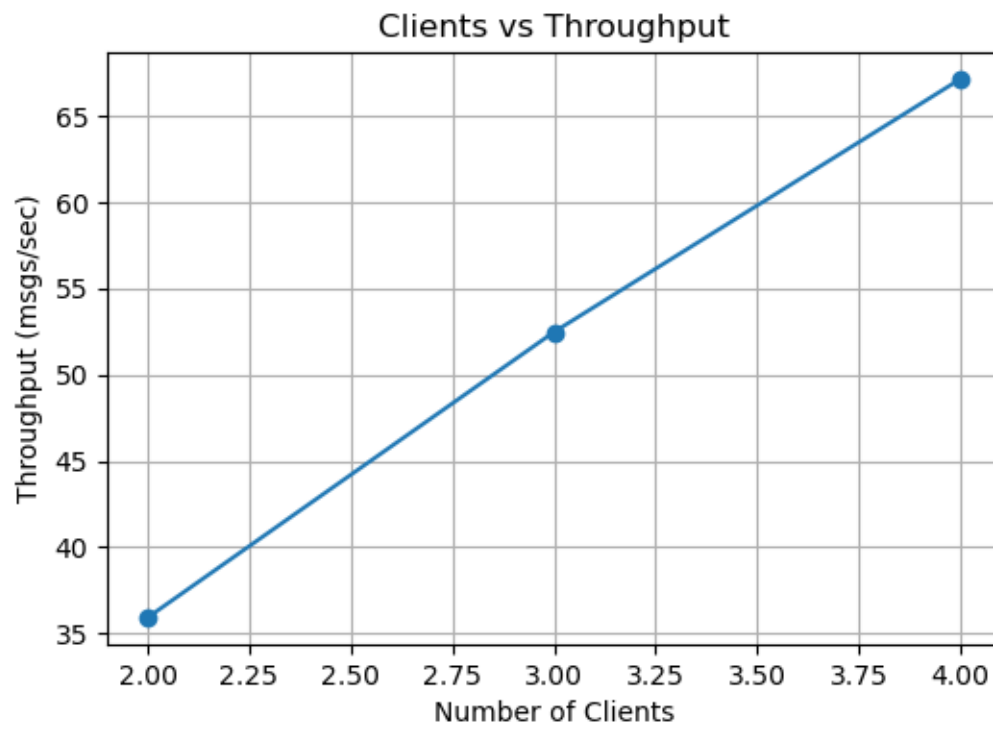
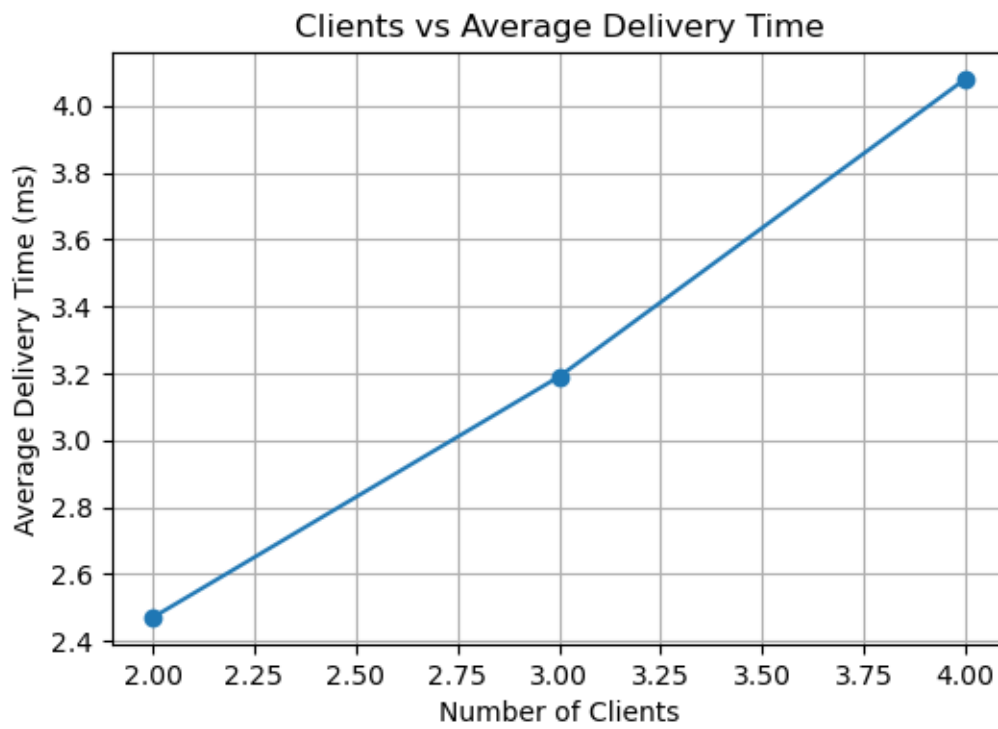
CPU utilization remained relatively stable during testing despite increasing client activity. The optimized connection management reduced unnecessary processing associated with disconnected clients and improved exception handling minimized repeated processing overhead.

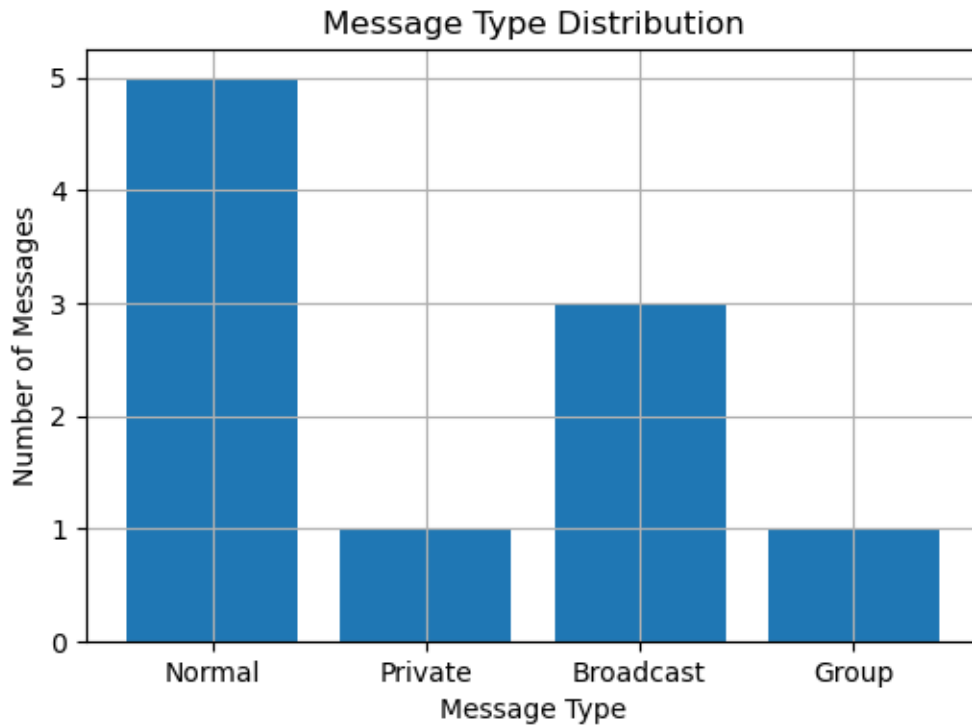
The results demonstrate efficient processor utilization while maintaining responsive application performance.

6.4 Memory Utilization Analysis

Memory consumption remained consistent throughout prolonged execution. Automatic client cleanup prevented disconnected users from occupying unnecessary memory, while proper socket management reduced resource leakage.

These improvements demonstrate enhanced long-term stability and indicate that the application can operate continuously without excessive memory growth.





7. Wireshark Verification

Wireshark was used to verify TCP communication on the loopback interface using the filter `tcp.port == 5000`. The captured traffic confirms the TCP three-way handshake, push/acknowledgement of application data, and connection teardown for every feature exercised.

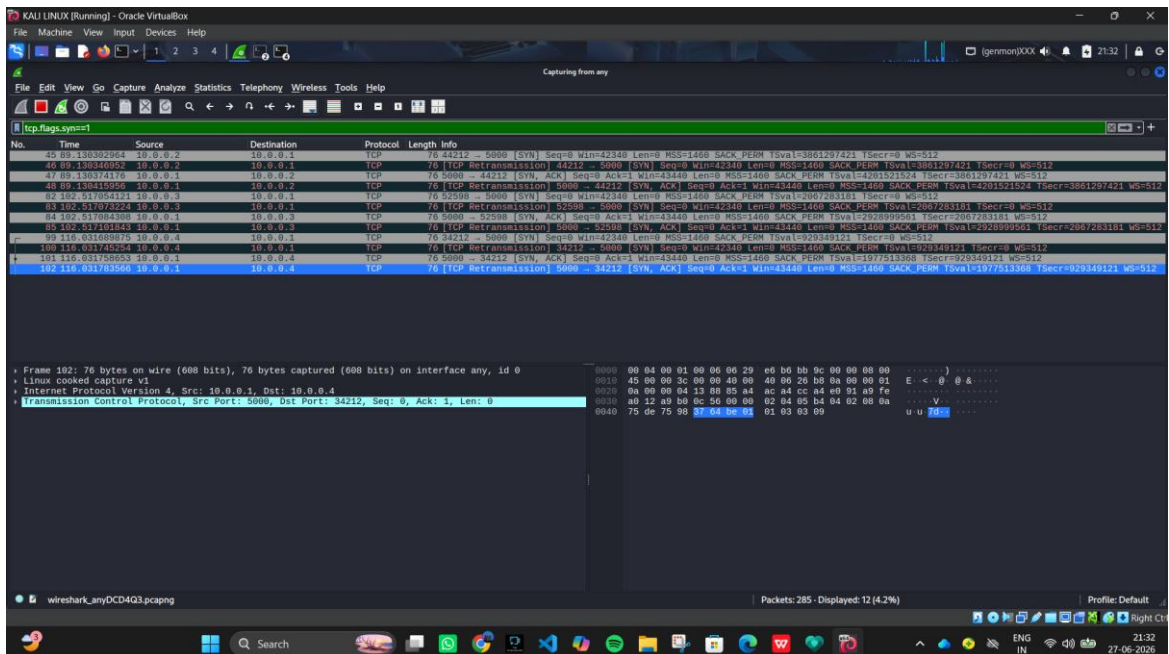
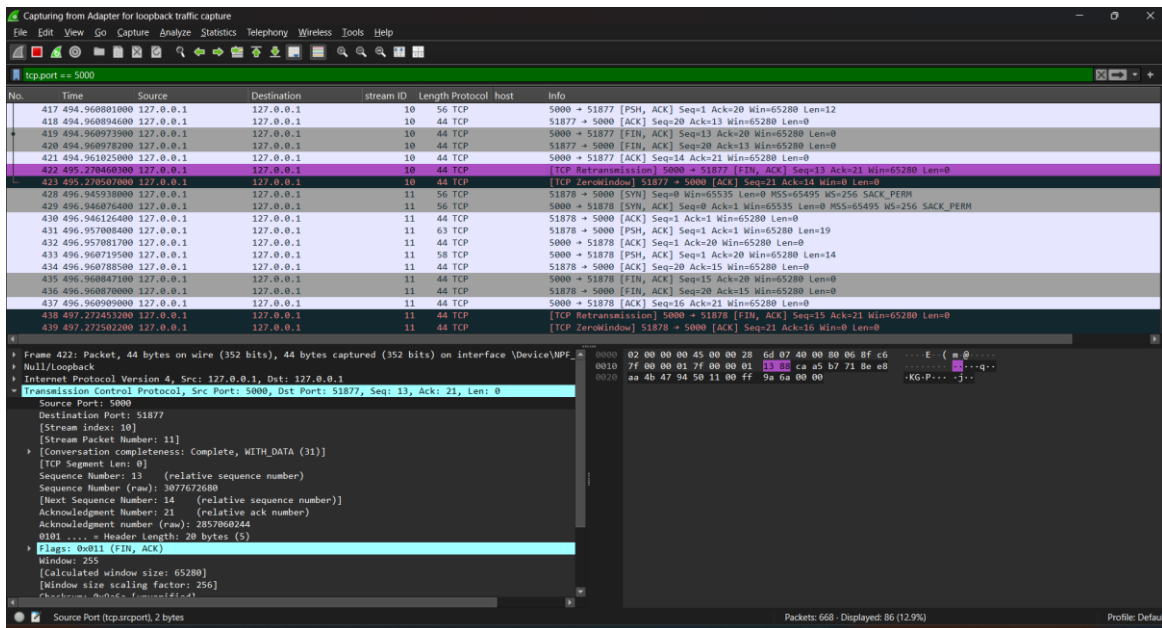
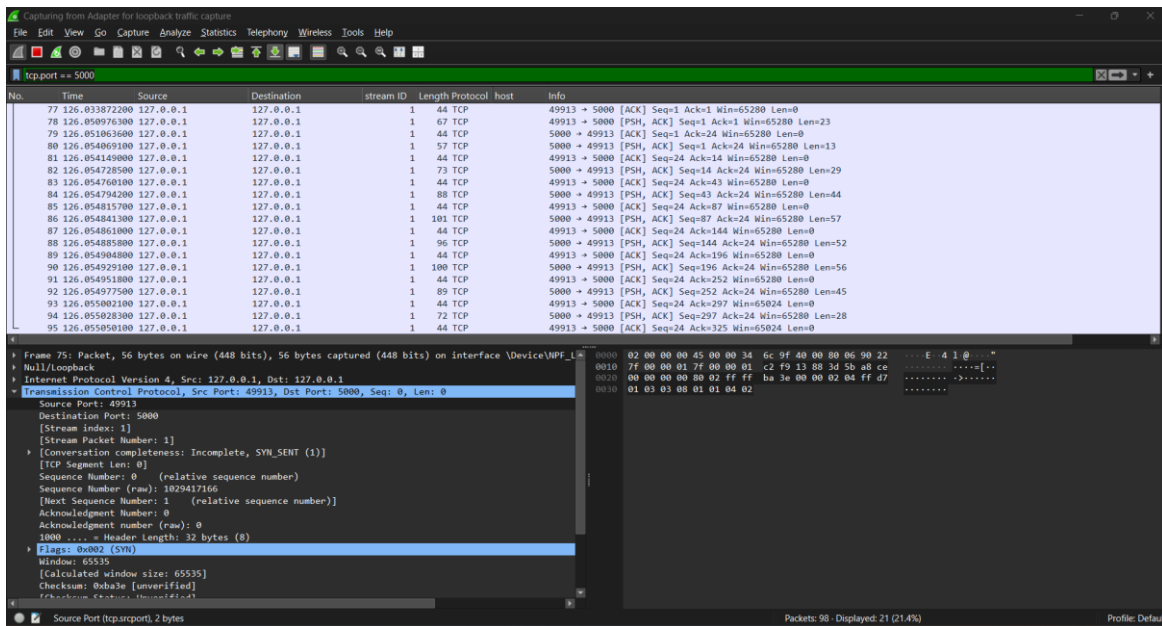


Fig 7.1 — TCP Three-Way Handshake on Port 5000



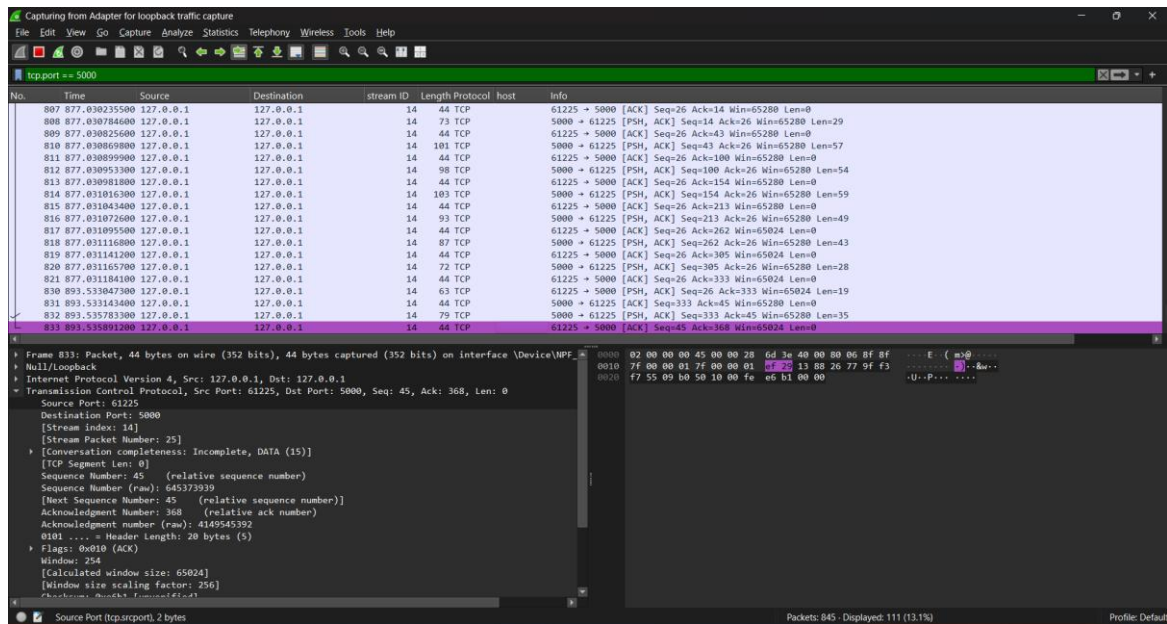


Fig 7.4 — Broadcast Message Traffic Capture

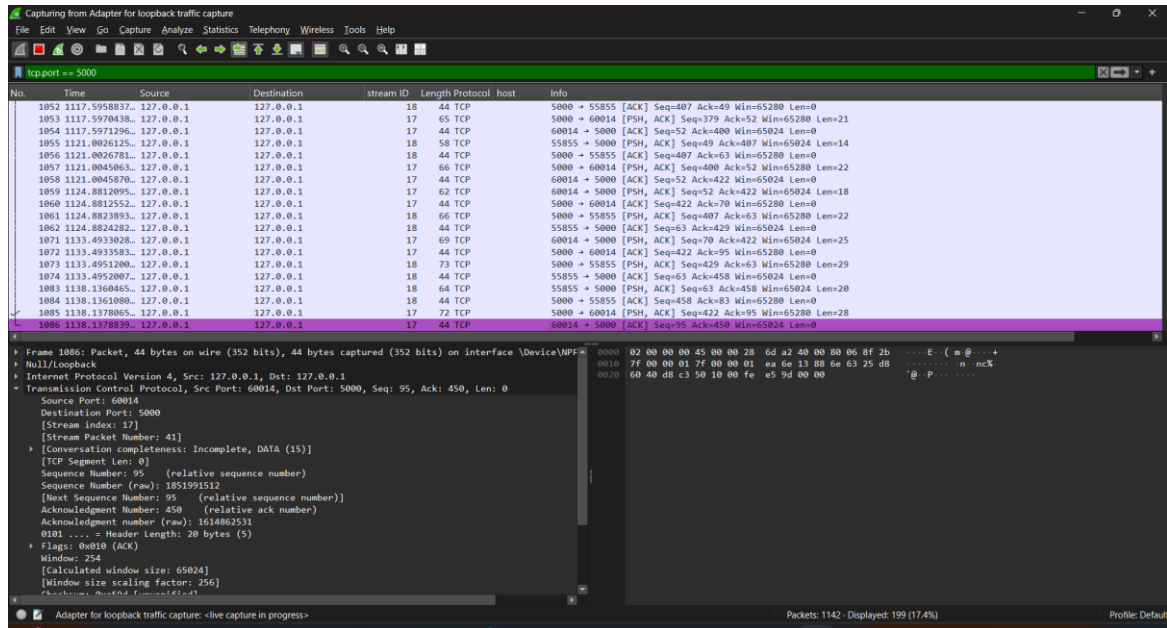


Fig 7.5 — Private Message Traffic Capture

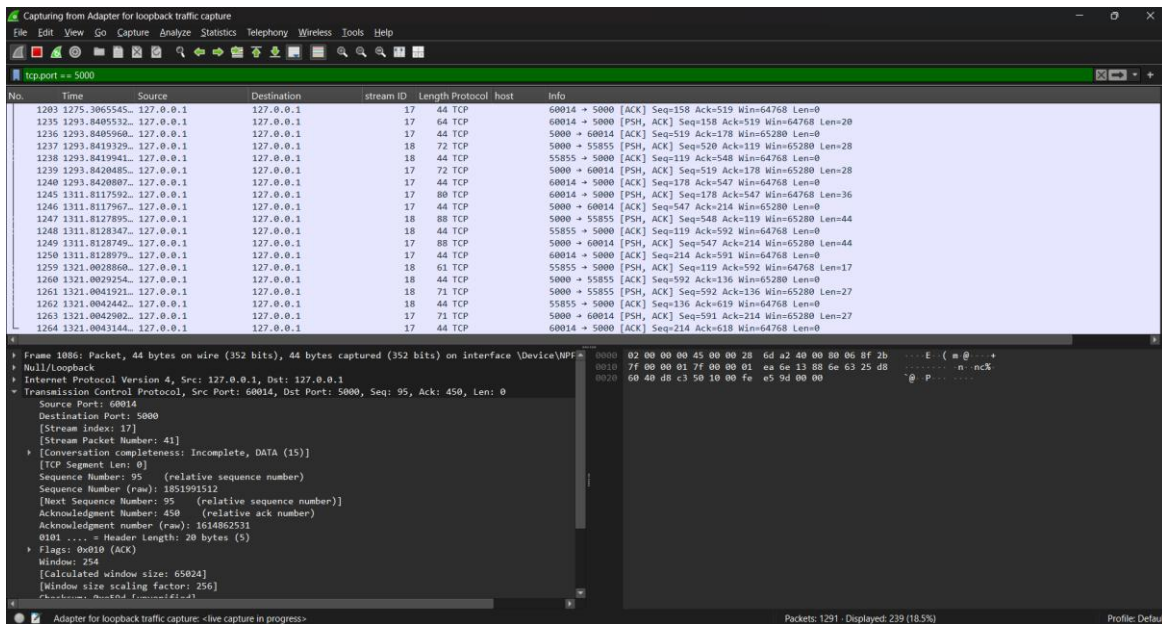


Fig 7.6 — Group Chat Traffic Capture

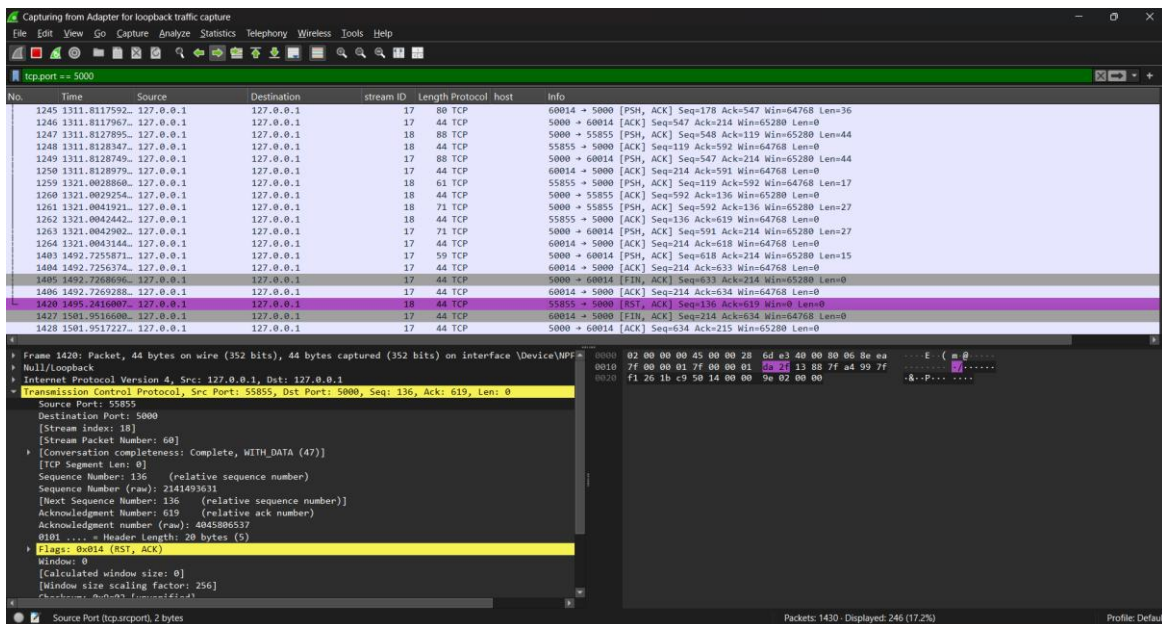


Fig 7.7 — Connection Termination (Disconnect) Capture

Across all captures, communication consistently occurs over TCP port 5000, confirming that every application feature — authentication, broadcast, private messaging, and group chat — is carried over the same secured TCP channel.

8. Conclusion

This assignment successfully transformed a basic TCP chat application into a secure communication platform. Security mechanisms such as password hashing, duplicate login prevention, failed login protection, session timeout, secure logging, and input validation significantly improved the application's reliability and security.

Wireshark verification confirmed successful TCP communication, while extensive testing demonstrated the correct functioning of all implemented features.

The project strengthened practical knowledge of TCP socket programming, authentication mechanisms, secure software development, and network packet analysis. Future improvements may include TLS encryption, database integration, role-based access control, and two-factor authentication.

9. References

- Python Official Documentation — <https://docs.python.org>
- Python Socket Programming Documentation
- Python hashlib Documentation
- Wireshark User Guide — <https://www.wireshark.org/docs/>
- ISEA Summer Internship Assignment 7 Specification

REFLECTION QUESTIONS

1. Which optimization produced the greatest improvement?

The optimization that produced the greatest improvement was implementing proper connection management and automatic client cleanup. Earlier, disconnected clients sometimes remained in memory, but after adding cleanup mechanisms and resource management, the server became more stable and could handle multiple clients reliably.

2. Why is scalability important?

Scalability is important because it allows an application to support an increasing number of users without affecting performance or stability. A scalable chat application can handle multiple simultaneous clients while maintaining smooth communication and efficient resource usage.

3. What reliability issues did you

discover?

During development, several reliability issues were identified, including unexpected client disconnections, socket resource leaks, unhandled exceptions, and improper cleanup after connection loss. These issues were resolved by implementing graceful shutdown, timeout handling, automatic reconnection, and better exception handling.

4. Which optimization was the most difficult?

The most difficult optimization was implementing automatic reconnection and proper connection management. It required careful handling of sockets, client cleanup, retry mechanisms, and exception handling to ensure that users could reconnect without causing application crashes or resource leaks.

5. What additional improvements are required to support IO

users?

To support 100 users, the application would require asynchronous networking or thread pooling, database-backed user management, TLS encryption, load balancing, optimized message queues, efficient memory management, and deployment on a dedicated server with higher hardware resources.